

产品需求文档 (PRD)

InfluxDB 可视化平台项目

刘子钰 · 2025 年 6 月

1. 文档信息

产品名称	InfluxDB 可视化平台项目
文档版本	v1.0
作者	刘子钰
日期	2025 年 6 月
项目背景	阿德莱德大学 Master of Computing and Innovation 课程项目
团队规模	8 人, Agile Scrum, 5 个双周 Sprint
技术栈	React + Vite · Node.js + Express · InfluxDB 2.7 · Grafana 9.5 · Docker · Redis

2. 产品背景

2.1 问题描述

InfluxDB 是工业级时序数据库, 广泛应用于 IoT 与运营监控场景。其原生查询语言 Flux 学习门槛高, 导致工厂管理人员、业务分析师等非技术用户无法自主获取数据。

每次数据需求都需依赖工程师介入, 导致平均响应周期长, 且需求频繁变化时效率极低, 工程师资源被占用在低价值的数据提取工作上。

2.2 解决方案

设计零代码数据查询与可视化平台, 使非技术用户无需学习和使用任何查询语法, 即可通过下拉选择与拖拽操作完成数据探索, 结果以图表形式实时呈现。

3. 目标用户

3.1 主要用户群体

用户类型	典型场景	核心诉求
工厂管理人员	定期提取数据生成报告	可查看生成的 Flux 语句, 保留透明度

3.2 用户痛点

- 无法独立完成数据查询，每次都需工程师协助，效率低
- Flux 语法复杂，学习成本高，非技术用户难以上手
- 数据探索路径不直观，不知道有哪些可查询的字段和指标
- 查询结果展示不够直观，难以快速洞察数据趋势

4. 功能需求

4.1 功能优先级列表

功能模块	优先级	说明
可视化查询构建器	P0	核心功能，下拉选择 Bucket / Measurement / Field / 时间范围
图表渲染	P0	支持折线图、柱状图、面积图、数据表格四种类型
单点登录双系统	P0	输入一次 Token，自动连接 InfluxDB 与 Grafana
View Flux 可选显示	P1	高级用户可查看生成的 Flux 语句，普通用户不感知
Data Tree 数据浏览	P1	可视化数据层级结构，支持拖拽节点至过滤区
AI 自然语言查询	P2	输入自然语言，AI 解析意图并生成 Flux 语句

4.2 核心功能说明

■ P0：可视化查询构建器

- 用户通过下拉菜单依次选择：Bucket（数据桶）→ Measurement（测量指标）→ Field（字段）→ 时间范围
- 每一步选择后，下一步选项自动根据上下文过滤，减少无效选项
- 点击「执行查询」后，系统在后端自动生成 Flux 语句并请求数据

■ P0：图表渲染

- 支持四种图表类型：折线图、柱状图、面积图、数据表格
- 用户可在查询结果页一键切换图表类型，无需重新查询
- 图表基于 Grafana 渲染，保证工业级展示质量

■ P0：单点登录双系统

- 用户只需输入一次 InfluxDB Token，系统后台自动完成 InfluxDB 与 Grafana 双平台鉴权
- 降低非技术用户的配置门槛，避免重复登录造成的困惑

■ P1: View Flux 可选显示

- 默认界面不显示 Flux 代码，保持简洁
- 高级用户点击「View Flux」按钮可查看系统自动生成的完整 Flux 查询语句
- 设计原则：透明度优先，用户可选择是否感知底层逻辑

■ P1: Data Tree 数据浏览

- 以树状结构可视化展示 InfluxDB 中的数据层级（Bucket → Measurement → Field）
- 用户可跨层级拖拽任意节点至过滤区，适合需要先探索数据结构的用户
- 与下拉选择器互补：下拉适合目标明确的用户，Data Tree 适合需要探索的用户

■ P2: AI 自然语言查询

- 用户输入自然语言描述，如「查看过去 24 小时的温度趋势」
- AI 解析用户意图，提取关键参数（时间范围、指标、字段），并展示解析结果供用户确认
- 系统自动生成对应 Flux 语句并执行查询，Flux 语句可见，保持透明度
- 呼应 Sprint 2 客户需求：支持简单与高级两种查询模式

5. 核心产品决策

决策一：下拉拖拽 vs 自然语言输入

背景：早期曾讨论直接让用户输入自然语言完成查询，交互更简单。

决策：优先采用下拉拖拽，将 AI 自然语言查询定为 P2 升级路径。

原因：

- 目标用户（工厂管理人员）对 AI 黑盒结果有可控性顾虑，需要清楚知道自己「选了什么」
- 下拉选择提供明确的选项边界，减少用户输入错误，降低认知负担
- AI 模式作为可选升级路径保留，面向更高级的用户场景

决策二：Flux 代码可选可见

背景：Sprint 2 客户明确要求「保持界面简洁，非程序员为核心用户」。

决策：设计「View Flux」按钮，默认不显示代码，按需展示。

原因：

- 普通用户完全感知不到 Flux 代码，界面保持简洁
- 高级用户（如工程师）可查看和调试生成的语句，满足透明度需求

- 一个设计同时服务两类用户，避免为不同角色开发两套界面

决策三：单次登录自动连接双系统

背景：系统依赖 InfluxDB 与 Grafana 两个平台，原本需要分别配置鉴权。

决策：用户只需输入一次 InfluxDB Token，后台自动完成双系统连接。

原因：

- 非技术用户对「Token」「鉴权」等概念陌生，双重配置极易出错
- 将技术复杂度封装在后端，用户只感知「登录成功」这一结果

决策四：Data Tree 与下拉选择配合

背景：用户查询路径分为两类——目标明确（直接选字段）和需要先探索数据结构。

决策：两种交互方式并存，互为补充，而非替代关系。

原因：

- 下拉选择适合目标明确的用户，操作路径短、效率高
- Data Tree 适合需要先了解数据结构的用户，可视化层级关系更直观
- 两者共存覆盖更广的用户场景，且不增加界面复杂度

6. 产品转折点

Sprint 4 客户反馈指出，团队过于专注后端实现，忽视了前端直觉体验。具体表现为：

- 查询构建器交互步骤繁琐，操作不够流畅
- 图表展示样式与用户预期有差距，过于单一
- 整体视觉设计不够简洁，信息层级不清晰

团队据此调整开发重心，从后端逻辑优化转向前端交互体验提升。这次转折让我深刻理解：

- 技术实现如果不能转化为可感知的用户体验，产品价值会大打折扣
- B 端工具的核心竞争力不仅是功能完整性，更是操作效率与使用舒适度
- 产品经理需要在每个 Sprint 持续校准开发方向，而不仅仅是在需求阶段定义功能

7. 成功指标

指标维度	衡量方式	目标
用户自主完成率	非技术用户无需工程师协助完成查询的比例	≥ 80%
查询成功率	用户发起查询后成功返回结	≥ 95%

	果的比例	
操作步骤数	完成一次基础查询所需点击步骤	≤ 5 步
工程师介入频率	用户提出数据需求后需工程师介入的比例	较上线前降低 60%
AI 模式采用率	使用 AI 自然语言查询的用户占比 (P2 上线后)	≥ 20%

8. 非功能需求

- 性能：查询响应时间 ≤ 3 秒 (Redis 缓存热点数据)
- 可用性：支持 Docker 一键部署，环境配置时间 ≤ 10 分钟
- 安全性：Token 不在前端明文存储，通过后端代理请求 InfluxDB
- 兼容性：支持主流浏览器 (Chrome、Firefox、Edge)
- 可扩展性：支持多 Bucket 接入，预留 AI 模式插件化扩展接口

9. 附录

Demo 地址：<https://zyl1study.github.io/influxdb-nocode>

GitHub：github.com/zyL1STUDY/influxdb-nocode

Demo 功能说明：

- 传统模式：下拉选择数据条件 → Data Tree 拖拽过滤标签 → 执行查询 → 切换图表类型
- AI 模式：输入自然语言 → AI 解析意图并展示参数 → 自动生成 Flux 语句 → 渲染图表